

1. Callback kya hota hai?

Simple definition

Callback = ek function jo hum kisi doosre function ko argument ke roop me pass karte hain, aur doosra function use baad me execute karta hai.

Example:

```
</> JavaScript

function greet(name, callback) {
  console.log("Hello " + name);

  callback();
}

function bye() {
  console.log("Bye!");
}

greet("Tarun", bye);
```

Flow:

```
greet()
  ↓
Hello Tarun
  ↓
callback()
  ↓
bye()
  ↓
Bye!
```

2. Function ko argument ki tarah kaise pass kar sakte hain?

JavaScript me functions **first-class citizens** hain.

Matlab function ko:

- variable me store kar sakte ho
- argument me pass kar sakte ho
- return kar sakte ho
- object/property me rakh sakte ho

Example:

```
</> JavaScript
function add(a, b) {
  return a + b;
}

let x = add;

console.log(x(10, 20));
```

3. Callback ka basic example

```
</> JavaScript
function calculate(a, b, operation) {
  let result = operation(a, b);

  console.log(result);
}

function add(x, y) {
  return x + y;
}

calculate(10, 20, add);
```

4. Anonymous callback

Callback ke liye alag function banana zaroori nahi.

</> JavaScript

```
function calculate(a, b, callback) {  
  console.log(callback(a, b));  
}  
  
calculate(10, 20, function (x, y) {  
  return x + y;  
});
```

Ya arrow function:

</> JavaScript

```
calculate(10, 20, (x, y) => {  
  return x + y;  
});
```

Short form:

`</>` JavaScript

```
calculate(10, 20, (x, y) => x + y);
```

5. Callback ka use kyu hua?

Yahan se actual important concept start hota hai.

JavaScript me kuch operations **immediately complete nahi hote**.

Example:

Database query
API request
File read
Timer
Network request



Suppose:

`</>` JavaScript

```
getUser();  
getPosts();
```

Agar `getUser()` ko 2 seconds lag rahe hain, to tum chahte ho:

getUser complete
↓
getPosts start

Iske liye callback use kiya ja sakta tha.

6. Asynchronous callback

Example:

```
</> JavaScript

console.log("Start");

setTimeout(() => {
  console.log("Hello");
}, 2000);

console.log("End");
```

Output:

```
Start
End
Hello
```

Yahan:

```
</> JavaScript

() => {
  console.log("Hello");
}
```

`setTimeout()` ko diya gaya callback hai.

JavaScript kehta hai roughly:

"2 seconds baad ye callback execute kar dena."



Isliye callback ka relation **asynchronous JavaScript + event loop** se bahut important hai.

8. Real-world example

Maan lo:



Har operation previous operation ke result par dependent hai.

Callback approach:

`</>` JavaScript

```
loginUser("tarun", function (user) {  
  
    getUserData(user.id, function (userData) {  
  
        getOrders(userData.id, function (orders) {  
  
            getOrderDetails(orders[0].id, function (order) {  
  
                getPayment(order.paymentId, function (payment) {  
  
                    console.log(payment);  
  
                });  
  
            });  
  
        });  
  
    });  
  
});  
  
});
```



9. Callback Hell kya hai?

Jab bahut saare dependent asynchronous callbacks ek doosre ke andar deeply nested ho jaate hain, to code pyramid/arrow shape jaisa dikhne lagta hai. Isse Callback Hell kehte hain.

Example:

`</>` JavaScript

```
doA(function () {  
  
    doB(function () {  
  
        doC(function () {  
  
            doD(function () {  
  
                doE(function () {  
  
                    console.log("Done");  
  
                });  
  
            });  
  
        });  
  
    });  
  
});
```



Structure dekho:

```
doA  
└─ doB  
   └─ doC  
      └─ doD  
         └─ doE
```

Ye **Pyramid of Doom** bhi kaha jata hai.

10. Callback Hell ki main problem kya hai?

Sirf indentation problem nahi hai.

Multiple problems hoti hain.

1. Readability kharab

`</>` JavaScript

```
A(() => {  
  B(() => {  
    C(() => {  
      D(() => {  
        E(() => {  
          }  
        }  
      }  
    }  
  }  
});
```

Samajhna difficult ho jata hai.



2. Maintainability difficult

Kal tumhe `c()` ke baad ek aur operation add karna hai:

A → B → C → New → D → E

Nested structure aur complicated ho jayega.

3. Error handling difficult

Suppose har function error de sakta hai:

```
</> JavaScript

getUser(function (err, user) {

    if (err) {
        // error
    }

    getOrders(user, function (err, orders) {

        if (err) {
            // error
        }

        getPayment(orders, function (err, payment) {

            if (err) {
                // error
            }

        });
    });
});
```



Error checking repeat hoti hai.

4. Debugging difficult

Agar:

A → B → C → D → E

me **D** fail ho gaya, to deeply nested code debug karna difficult ho sakta hai.

11. Error-first callback pattern

Node.js me traditional callbacks me commonly ye pattern milta hai:

❏ JavaScript

```
function operation(callback) {  
  
    // success  
    callback(null, result);  
  
}
```

Error:

❏ JavaScript

```
callback(error, null);
```

Async_await_part_1

JavaScript

```
function divide(a, b, callback) {  
  
    if (b === 0) {  
        callback(new Error("Cannot divide by zero"));  
        return;  
    }  
  
    callback(null, a / b);  
}
```

Use:

JavaScript

```
divide(10, 2, (err, result) => {  
  
    if (err) {  
        console.log(err.message);  
        return;  
    }  
  
    console.log(result);  
});
```



15. Callback vs Callback Hell

Concept	Meaning
Callback	Function passed to another function
Callback function	Function executed later/by another function
Async callback	Callback executed after async operation completes
Callback chain	Multiple callbacks executed sequentially
Callback hell	Excessive/deeply nested callback chain
Pyramid of Doom	Callback hell ka visual structure
Error-first callback	<code>(err, result)</code> pattern
Promise	Callback-based async code ka cleaner abstraction
async/await	Promise-based code ko readable banane ka syntax



13. Same Callback Hell → Promise

Callback version:

</> JavaScript

```
getUser(function (err, user) {  
  
    if (err) return console.log(err);  
  
    getOrders(user.id, function (err, orders) {  
  
        if (err) return console.log(err);  
  
        getPayment(orders[0].id, function (err, payment) {  
  
            if (err) return console.log(err);  
  
            console.log(payment);  
  
        });  
  
    });  
  
});
```



Promise version:

</> JavaScript

```
getUser()
  .then(user => {
    return getOrders(user.id);
  })
  .then(orders => {
    return getPayment(orders[0].id);
  })
  .then(payment => {
    console.log(payment);
  })
  .catch(err => {
    console.log(err);
  });
```

Much flatter

14. Promise → async/await

Modern version:

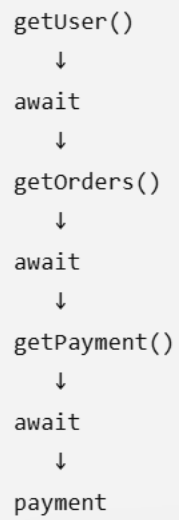
 JavaScript

```
async function process() {  
  
  try {  
  
    const user = await getUser();  
  
    const orders = await getOrders(user.id);  
  
    const payment = await getPayment(orders[0].id);  
  
    console.log(payment);  
  
  } catch (err) {  
  
    console.log(err);  
  
  }  
}
```



Flow visually:

```
getUser()  
  ↓  
await  
  ↓  
getOrders()  
  ↓  
await  
  ↓  
getPayment()  
  ↓  
await  
  ↓  
payment
```



Readable almost like synchronous code.

Haan, **callback** ki zarurat exactly kyu padi, ye samajhne ke liye ek p

1. Problem kya thi?

Maan lo hume user ka data server se lana hai:

JavaScript

```
const user = getUser();  
console.log(user);
```

Problem: **data turant nahi milta.**

Server/database/network ko time lag sakta hai:

Request bhejo
↓
Server process kare
↓
Database se data aaye
↓
Response mile



Is dauran JavaScript ko **poora program rok dena** practical nahi hai.

2. Agar JavaScript wait karne lage?

Suppose:

```
</> JavaScript

console.log("Start");

const user = getUser(); // 5 sec

console.log(user);

console.log("End");
```

Agar `getUser()` 5 seconds leta hai aur JS wahi ruk jaye, to:

Start



WAIT 5 seconds ❌



user



End



Agar server par 1000 requests aa rahi hain, to aise wait karna bahut inefficient hoga.

JavaScript ka idea tha:

"Main wait nahi karunga. Jab operation complete ho jaye, mujhe bata dena ki ab kya karna hai."

Yahin **callback** ka use aaya.

3. Callback ne problem solve kaise ki?

Hum kehte hain:

JavaScript

```
getUser(function (user) {  
  console.log(user);  
});
```

Meaning:

"getUser, tum apna kaam karo. Jab user ka data mil jaye, tab ye function chala dena."

Flow:

```
getUser()  
↓  
Database/Server  
↓  
wait...  
↓  
data received  
↓  
callback(user)  
↓  
console.log(user)
```

JavaScript meanwhile doosra kaam kar sakta hai.

4. Real-life example

Restaurant me order diya.

Without callback:

```
Order diya
↓
Counter ke saamne khade raho
↓
Food banta rahe
↓
5 min wait
↓
Food mila
↓
Ab doosra kaam
```

Callback:

```
Order diya
↓
"Food ready ho to mujhe bula dena"
↓
Tum table par chale gaye
↓
Tum doosra kaam karte ho
↓
Food ready
↓
Restaurant tumhe bulaata hai
```

"Food ready ho to mujhe bula dena" = callback

5. Callback ka core purpose

Callback ka main purpose ye nahi hai ki:

"Function ke andar function likhna."

Actual purpose:

Future me hone wale event/operation ke complete hone par kya karna hai, ye pehle se define karna.

Example:

```
</> JavaScript
downloadFile("photo.jpg", function () {
  console.log("Download complete");
});
```

Tumne bola:

```
File download karo
  ↓
jab complete ho
  ↓
callback chala dena
```

6. Asynchronous programming me callback important kyu tha?

Node.js jaise environments me bahut operations I/O based hote hain:

```
File
Database
Network
API
HTTP request
Timer
```



Example:

```
</> JavaScript

fs.readFile("data.txt", "utf8", function (err, data) {
  console.log(data);
});
```

Yahan:

```
</> JavaScript

function (err, data) {
  console.log(data);
}
```

callback hai.

7. Sabse important: Callback ne kya achieve kiya?

Without callback

```
Operation
↓
WAIT
↓
Result
↓
Next work
```

Callback ke saath

```
Operation
↓
Callback register
↓
JavaScript doosra kaam kar sakti hai
↓
Operation complete
↓
Callback execute
```

Isliye callback ka connection **non-blocking/asynchronous programming** se bahut important hai.

8. Lekin callback ki problem bhi hui

Ek operation:

```
</> JavaScript

getUser(function (user) {
  console.log(user);
});
```

Theek hai.

Lekin agar operations dependent hote gaye:

```
Get User
↓
Get Orders
↓
Get Product
↓
Get Payment
↓
Send Email
```



To callbacks nested hone lage:

</> JavaScript

```
getUser(function (user) {  
  
    getOrders(user.id, function (orders) {  
  
        getProduct(orders[0], function (product) {  
  
            getPayment(product, function (payment) {  
  
                sendEmail(payment, function () {  
  
                    console.log("Done");  
  
                });  
  
            });  
  
        });  
  
    });  
  
});  
  
});
```



Callback Hell / Pyramid of Doom

Aur isi problem ko solve karne ke liye later **Promises** aur phir **async/await** popular hue.

Ek line me pura concept

Asynchronous kaam hai
↓
JavaScript ko unnecessarily block nahi karna
↓
"Kaam complete ho to ye function chala dena"
↓
Callback
↓
Bahut callbacks nested ho gaye
↓
Callback Hell
↓
Promises
↓
async/await



1. Promise kya hai?

Promise ek JavaScript object hai jo future me milne wale asynchronous operation ke result ko represent karta hai.

Simple:

Aaj:
"Result abhi nahi mila"

Future:
"Result mil gaya" → Success
ya
"Kaam fail ho gaya" → Error



Example:

```
</> JavaScript  
  
const promise = new Promise((resolve, reject) => {  
  
    // async operation  
  
    resolve("Success");  
  
});
```

Yahan Promise kehta hai:

"Main abhi result nahi de raha, lekin future me ya to success dunga ya failure."

2. Promise ki 3 states

Promise ki **3 states** hoti hain:

State	Meaning
pending	Kaam abhi complete nahi hua
fulfilled	Kaam successfully complete
rejected	Kaam fail ho gaya

3. resolve() aur reject()

Promise banate waqt:

```
JavaScript
const p = new Promise((resolve, reject) => {
    resolve("Data mil gaya");
});
```

resolve()

Success:

```
JavaScript
resolve(data);
```

`reject()`

Failure:

`</> JavaScript`

```
reject(error);
```

Example:

`</> JavaScript`

```
const p = new Promise((resolve, reject) => {  
  
  let success = true;  
  
  if (success) {  
    resolve("User mil gaya");  
  } else {  
    reject("User nahi mila");  
  }  
  
});
```



4. Promise ka result kaise receive kare?

Promise ke saath mainly:

```
⟨/⟩ JavaScript
```

```
.then()  
.catch()  
.finally()
```

use hote hain.

Success:

```
⟨/⟩ JavaScript
```

```
p.then((result) => {  
    console.log(result);  
});
```

Error:

```
</> JavaScript  
  
p.catch((error) => {  
    console.log(error);  
});
```

Complete:

```
</> JavaScript  
  
p  
    .then((result) => {  
        console.log(result);  
    })  
    .catch((error) => {  
        console.log(error);  
    });
```

5. Promise ka real purpose

Callback me:

```
</> JavaScript

getUser(function (user) {
  console.log(user);
});
```

Agar multiple dependent operations aa gaye:

```
getUser
  ↓
getOrders
  ↓
getPayment
  ↓
sendEmail
```

to:

```
Callback
  ↓
Nested callback
  ↓
Callback Hell
```

Promise ne code ko **flat chain** me likhne ka way diya:

```
</> JavaScript

getUser()
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders[0].id))
  .then(payment => sendEmail(payment))
  .catch(error => console.log(error));
```

6. Promise Chaining kya hai?

Ye bahut important hai.

Ek Promise ke `.then()` se returned Promise/value ko next `.then()` me pass karke sequential operations perform karna = Promise Chaining.

Example:

`</>` JavaScript

```
Promise.resolve(10)
  .then(value => {
    return value * 2;
  })
  .then(value => {
    return value + 5;
  })
  .then(value => {
    console.log(value);
  });
```

Flow:

```
Promise → 10
      ↓
    × 2
      ↓
    20
      ↓
    + 5
      ↓
    25
```

Output:

25

7. `.then()` ka sabse important rule

Suppose:

`</> JavaScript`

```
Promise.resolve(10)
  .then(value => {
    return value * 2;
  })
  .then(value => {
    console.log(value);
  });
```

Pehle `.then()` ne:

`</> JavaScript`

```
return 20;
```

kiya.

Ye `20` next `.then()` ko milta hai.

```
.then()
↓
return 20
↓
next .then(20)
```

Isi wajah se chaining possible hai.

8. Agar `.then()` me Promise return kare?

Ye aur important hai.

`</>` JavaScript

```
getUser()  
  .then(user => {  
    return getOrders(user.id);  
  })  
  .then(orders => {  
    console.log(orders);  
  });
```

Yahan:

JavaScript

```
return getOrders(user.id);
```

ek **Promise** return kar raha hai.

Next `.then()` tab chalega jab `getOrders()` ka Promise resolve hoga.

Flow:

```
getUser()  
  ↓  
Promise resolve  
  ↓  
user  
  ↓  
getOrders(user.id)  
  ↓  
Promise resolve  
  ↓  
orders  
  ↓  
next .then()
```



9. `.catch()` ka benefit

Promise chain me error handling easy hoti hai:

`</>` JavaScript

```
getUser()
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders))
  .then(payment => sendEmail(payment))
  .catch(error => {
    console.log(error);
  });
```

Agar:

getUser ❌

ya

getOrders ❌



sendEmail ❌

me error aaya, to `.catch()` handle kar sakta hai.

10. Callback Hell vs Promise Chaining

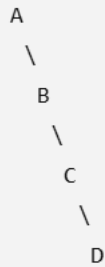
Callback Hell

JavaScript

```
getUser(function(user) {  
  
  getOrders(user.id, function(orders) {  
  
    getPayment(orders, function(payment) {  
  
      sendEmail(payment, function() {  
  
        console.log("Done");  
  
      });  
  
    });  
  
  });  
  
});
```



Shape:



Promise

JavaScript

```
getUser()  
  .then(user => getOrders(user.id))  
  .then(orders => getPayment(orders))  
  .then(payment => sendEmail(payment))  
  .then(() => console.log("Done"))  
  .catch(err => console.log(err));
```

```
A → B → C → D → Done
      ↓
    catch
```

Much easier to read.

11. Promise chaining me `return` kyu important hai?

Ye mistake beginners bahut karte hain.

Correct:

JavaScript

```
getUser()
  .then(user => {
    return getOrders(user.id);
  })
  .then(orders => {
    console.log(orders);
  });
```



Incorrect:

JavaScript

```
getUser()
  .then(user => {
    getOrders(user.id);
  })
  .then(orders => {
    console.log(orders);
  });
```

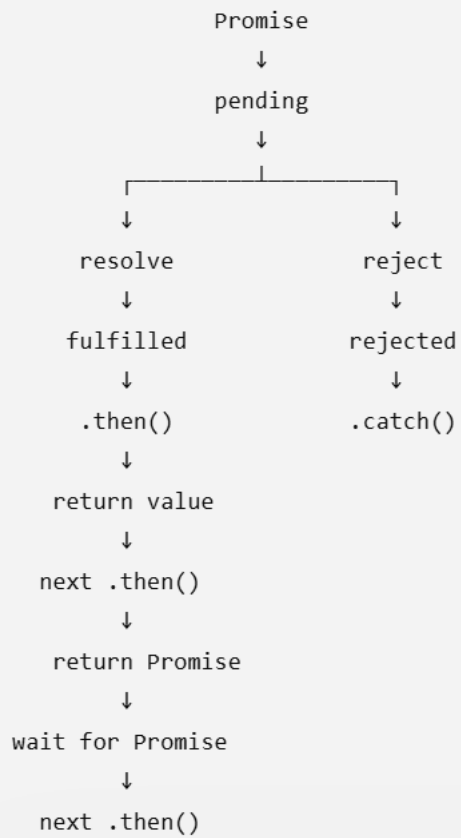
Kyun?

Kyuki first `.then()` ne `getOrders()` ka Promise **return hi nahi kiya**.

Rule yaad rakho:

Next `.then()` ko previous `.then()` ka returned value/Promise milta hai.

12. Promise ka complete mental model



Promise chaining

</> JavaScript

```
getUser()  
  .then(user => getOrders(user.id))  
  .then(orders => getPayment(orders))  
  .then(payment => console.log(payment))  
  .catch(err => console.log(err));
```

async/await

</> JavaScript

```
async function process() {  
  
  try {  
  
    const user = await getUser();  
    const orders = await getOrders(user.id);  
    const payment = await getPayment(orders);  
  
    console.log(payment);
```



```

    } catch (err) {
      console.log(err);
    }
  }
}

```

`async/await` actually **Promises ko replace nahi karta**; ye Promises ke upar cleaner syntax provide karta hai.

🔥 Exam/Interview ke liye 5 lines

Concept	Meaning	📄
Promise	Future asynchronous result ko represent karta hai	
Pending	Result abhi nahi aaya	
Fulfilled	Operation successful	
Rejected	Operation failed	

1. `async/await` ki zarurat kyu padi?

Promise chaining:

```

<> JavaScript
getUser()
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders))
  .then(payment => console.log(payment))
  .catch(err => console.log(err));

```

Ye callback hell se better hai, lekin multiple operations hone par chain lambi ho sakti hai.

Ye callback hell se better hai, lekin multiple operations hone par chain lambi ho sakti hai.

Isko readable banane ke liye:

```
</> JavaScript   
  
async function process() {  
  const user = await getUser();  
  const orders = await getOrders(user.id);  
  const payment = await getPayment(orders);  
  
  console.log(payment);  
}
```

Ye almost normal sequential code jaisa dikhta hai.

2. `async` kya karta hai?

`async` kisi function ke aage lagate hain:

```
</> JavaScript  
  
async function test() {  
  
}
```

Important: `async` function hamesha Promise return karta hai.

Example:

```
</> JavaScript  
  
async function test() {  
  return "Hello";  
}
```

Actually:

```
</> JavaScript
```

```
test();
```

Promise return karega:

```
Promise → fulfilled → "Hello"
```

Isliye:

```
</> JavaScript
```

```
test().then(result => {  
  console.log(result);  
});
```

Output:

```
Hello
```

CHAPTER 002

```
async function  
↓  
always returns Promise
```



3. `await` kya karta hai?

`await` kisi **Promise ke result ka wait** karne ke liye use hota hai.

```
</> JavaScript  
  
const result = await somePromise();
```



Meaning:

"Is Promise ka result aane tak is async function ki execution ko yahan temporarily pause karo, result aane par aage continue karo."

JavaScript

```
async function getData() {  
  
    const result = await fetchData();  
  
    console.log(result);  
  
}
```

Flow:

```
getData()  
↓  
fetchData()  
↓  
Promise pending  
↓  
async function yahan pause  
↓  
JavaScript doosra kaam kar sakti hai  
↓  
↓  
async function yahan pause  
↓  
JavaScript doosra kaam kar sakti hai  
↓  
Promise fulfilled  
↓  
result  
↓  
next line execute
```

Important: `await` poori JavaScript ko block nahi karta. Sirf us async function ki execution ko suspend karta hai.

4. `await` ke bina

`</>` JavaScript

```
function getData() {  
    return Promise.resolve("Data");  
}  
  
async function test() {  
  
    const result = getData();  
  
    console.log(result);  
}
```

`result` me Promise milega:

```
Promise { "Data" }
```

5. await ke saath

```
</> JavaScript  
  
async function test() {  
  
    const result = await getData();  
  
    console.log(result);  
}
```

Ab:

```
getData()  
↓  
Promise  
↓  
await  
↓  
resolved value  
↓  
"Data"
```

So:

```
</> JavaScript  
  
const result = await getData();
```

ka practical meaning hai:

Promise resolve hone ke baad uski actual value `result` me do.

6. `async` और `await` ka relationship

Dono ko pair ki tarah samjho:

`</>` JavaScript

```
async function test() {  
  
    const data = await getData();  
  
}
```

- `async` → function ko Promise-based banata hai
- `await` → Promise ke resolved result ko conveniently receive karta hai

Generally `await` ko **async function ke andar** use kiya jata hai. Modern JavaScript me modules ke top-level par bhi `await` allowed hai.

7. Error handling — try/catch

Promise me:

JavaScript

```
getData()  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

async/await me:

JavaScript

```
async function test() {  
  
  try {  
  
    const data = await getData();  
  
    console.log(data);  
  
  } catch (err) {  
  
    console.log(err);  
  }  
}
```



Agar:

```
</> JavaScript
```

```
getData()
```

Promise reject karta hai, to control:

```
await  
↓  
reject  
↓  
catch
```

me chala jata hai.

8. Real example

Maan lo:

```
User fetch karo
  ↓
Orders fetch karo
  ↓
Payment fetch karo
```

Promise chaining:

JavaScript

```
getUser()
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders))
  .then(payment => {
    console.log(payment);
  })
  .catch(err => {
    console.log(err);
  });
```



async/await:

</> JavaScript

```
async function process() {  
  
  try {  
  
    const user = await getUser();  
  
    const orders = await getOrders(user.id);  
  
    const payment = await getPayment(orders);  
  
    console.log(payment);  
  
  } catch (err) {  
  
    console.log(err);  
  
  }  
}
```

Read karna easy:

```
User lao  
↓  
Orders lao  
↓  
Payment lao  
↓  
Print karo
```

9. Ek bahut important misconception

✗ `await` ka matlab:

"JavaScript पूरी तरह रुक गयी."

Galat.

✓ **Actual:**

"`await` current async function ki execution ko temporarily suspend karta hai; JavaScript event loop दूसरे tasks handle kar sakta hai."

```
async function test() {  
  
  console.log("A");  
  
  await new Promise(resolve => {  
    setTimeout(resolve, 2000);  
  });  
  
  console.log("B");  
}  
  
test();  
  
console.log("C");
```

Output:

```
A  
C  
B
```

Kyun?

```
test()
↓
A
↓
await → function temporarily suspend
↓
C execute
↓
2 sec complete
↓
B
```

10. `await` sirf Promise ke liye?

Practically `await` kisi bhi value ke saath use ho sakta hai:

`</>` JavaScript

```
const x = await 10;
console.log(x);
```

Result:

10

Lekin iska real benefit **Promises** ke saath hota hai.

11. Sequential vs Parallel — important

Ye:

JavaScript

```
const user = await getUser();  
const products = await getProducts();
```

sequential hai.

```
getUser  
  ↓ complete  
getProducts  
  ↓ complete
```

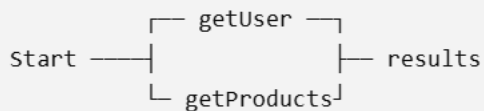
Agar dono independent hain, to parallel kar sakte ho:

Agar dono independent hain, to parallel kar sakte ho:

JavaScript

```
const [user, products] = await Promise.all([  
  getUser(),  
  getProducts()  
]);
```

Flow:



Isse unnecessary waiting kam ho sakti hai.

12. Callback → Promise → async/await

Pura evolution yaad rakho:

```
Callback
  ↓
Callback Hell
  ↓
Promise
  ↓
Promise Chaining
  ↓
async/await
```

Callback

```
</> JavaScript

getUser(function(user) {
  console.log(user);
});
```

Promise

```
</> JavaScript

getUser()
  .then(user => console.log(user))
  .catch(err => console.log(err));
```

async/await

```
</> JavaScript

async function test() {

  try {
    const user = await getUser();
    console.log(user);
  } catch (err) {
    console.log(err);
  }

}
```

Keywords	Meaning
async	Function ko Promise-returning function banata hai
await	Promise settle hone tak current async function ko temporarily pause karta hai
try	Async code run karna
catch	Rejected Promise/error handle karna
Promise.all()	Multiple independent Promises ko parallel wait karna